ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
**«Колледж связи № 54» имени П.М. Вострухина**

# ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №4

«Интеграция Flask и SQLite (чтение данных)»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

# Практическая работа №4. Интеграция Flask и SQLite (чтение данных)

**Ссылка на репозиторий:** https://github.com/gangdexyuhi/client-server-practice
**Папка работы:** practice_04_client_server/

## Цель работы

Подключить базу данных SQLite к Flask-приложению, вывести список задач из базы на веб-страницу с помощью шаблонизатора Jinja2 и разобраться в назначении sqlite3.Row.

## Используемые технологии

- Python 3.13, Flask 3.1
- Шаблонизатор Jinja2: {{ }}, {% for %}, {% if %}
- SQLite, sqlite3.Row (доступ к полям по имени)
- HTML и CSS

## Структура проекта

```
src/
├── init_db.py, check_db.py, crud_console.py
├── app.py                # чтение задач и передача в шаблон
├── app.db
├── requirements.txt
└── templates/
    └── index.html        # шаблон списка задач
```

## Описание реализации

Функция get_connection() открывает соединение и устанавливает conn.row_factory = sqlite3.Row, чтобы строки результата были доступны по имени колонки. Маршрут / выполняет SELECT, закрывает соединение (commit не нужен, так как данные не меняются) и вызывает render_template('index.html', items=rows).

```
import sqlite3
from flask import Flask, render_template

app = Flask(__name__)

DB_PATH = 'app.db'


def get_connection():
    """Соединение с базой. row_factory позволяет обращаться к полям по имени:
row['title']."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn


@app.route('/')
def index():
    # 1. подключение
    conn = get_connection()
    cursor = conn.cursor()

    # 2. чтение всех задач
    cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
    rows = cursor.fetchall()

    # 3. закрываем соединение (commit не нужен - данные не меняются)
    conn.close()

    # 4. передаём данные в шаблон под именем items
    return render_template('index.html', items=rows)


if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

В шаблоне index.html условие {% if items %} выбирает, показывать ли список или сообщение «Список пуст», цикл {% for item in items %} выводит задачи, а условие внутри атрибута class добавляет класс done выполненным задачам — они отображаются зачёркнутыми.

```
<h1>Мой список задач</h1>

    {# если в базе есть задачи - выводим список, иначе сообщение #}
    {% if items %}
        <ul>
            {% for item in items %}
                <li class="{% if item.status == 'done' %}done{% endif %}">
                    <strong>#{{ item.id }}</strong>
                    {{ item.title }}
                    <em>[{{ item.status }}]</em>
                </li>
            {% endfor %}
        </ul>
    {% else %}
        <p class="empty">Список пуст. Добавь первую задачу!</p>
    {% endif %}
```

## Ход работы и результаты запуска

1. Наполнил базу тестовыми задачами через crud_console.py: «Купить хлеб», «Сделать уроки» (отмечена выполненной), «Позвонить маме».

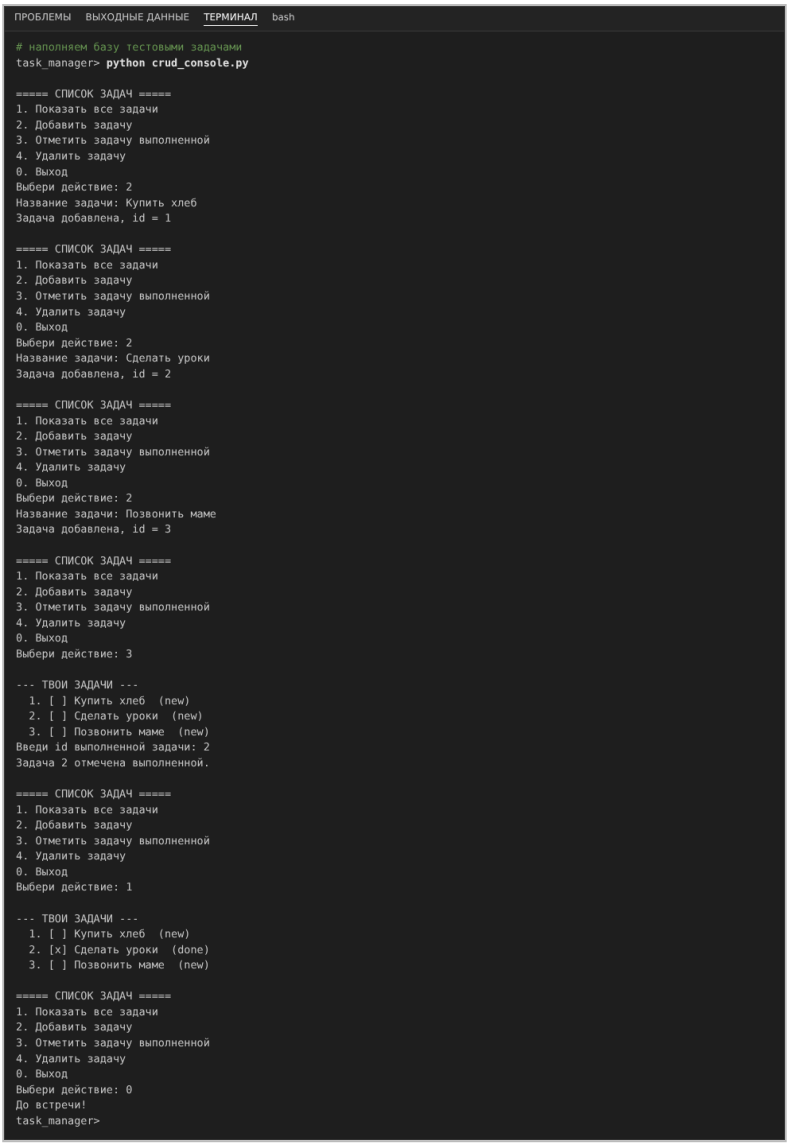



Рисунок 1 — Наполнение базы через консольное приложение

2. Запустил сервер и открыл главную страницу — задачи выводятся из базы, выполненная задача зачёркнута.

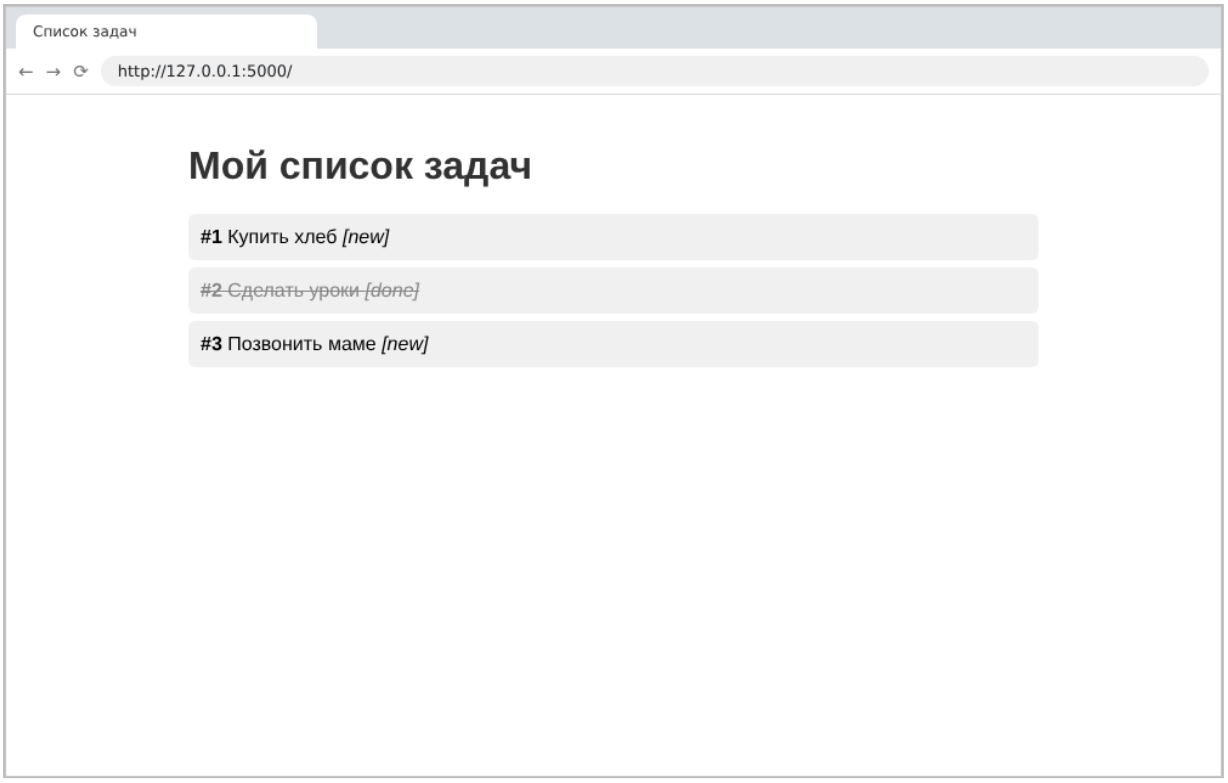



Рисунок 2 — Список задач из базы данных на веб-странице

3. Проверил «живую» связь с базой: не останавливая сервер, во втором терминале добавил задачу «Проверка связи», затем обновил страницу — новая задача появилась.

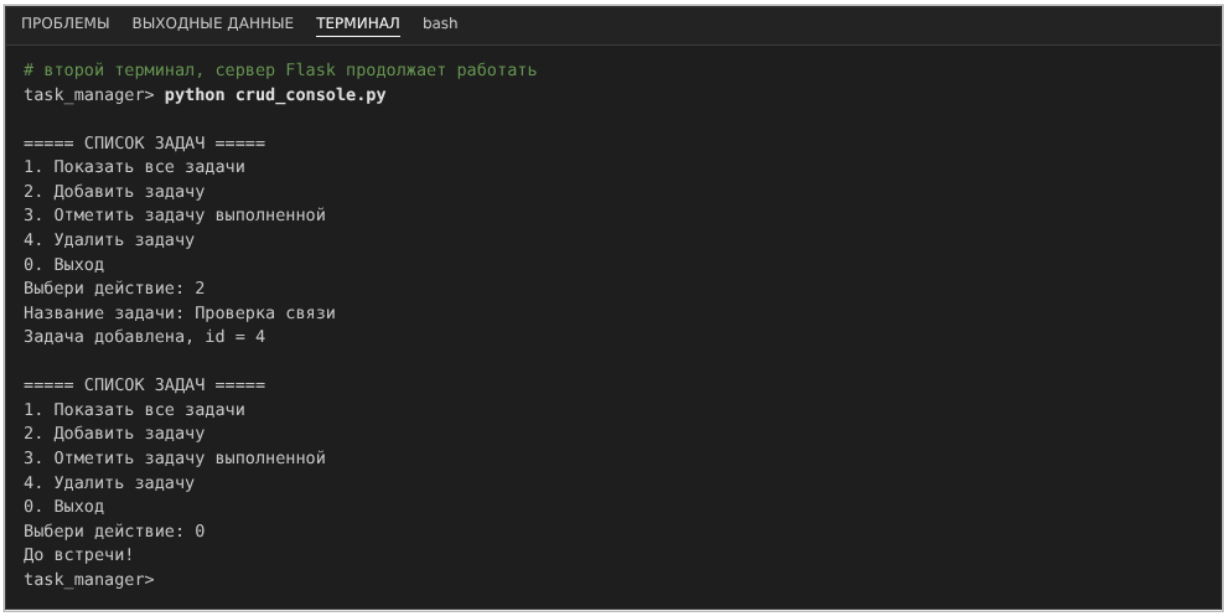



Рисунок 3 — Добавление задачи из второго терминала

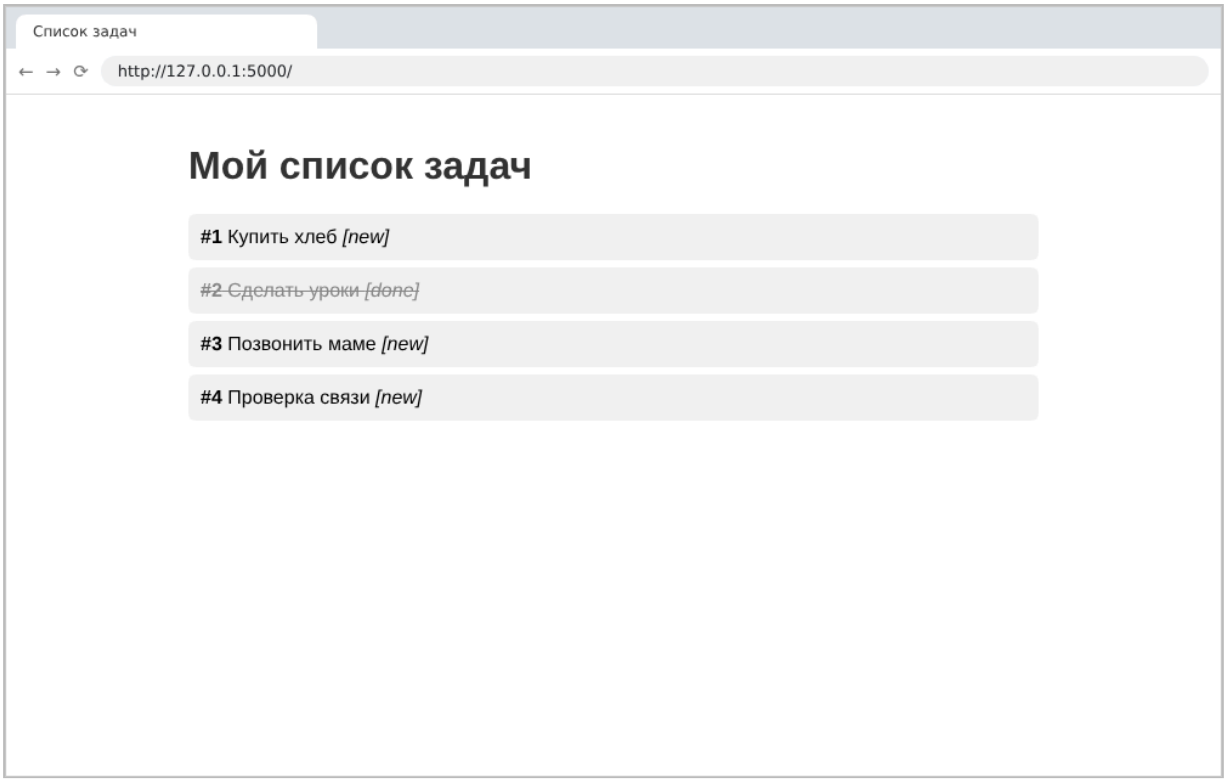



Рисунок 4 — Новая задача на странице после обновления

4. Удалил все задачи из базы и обновил страницу — сработала ветка {% else %}, выводится сообщение «Список пуст». Затем вернул задачи обратно.

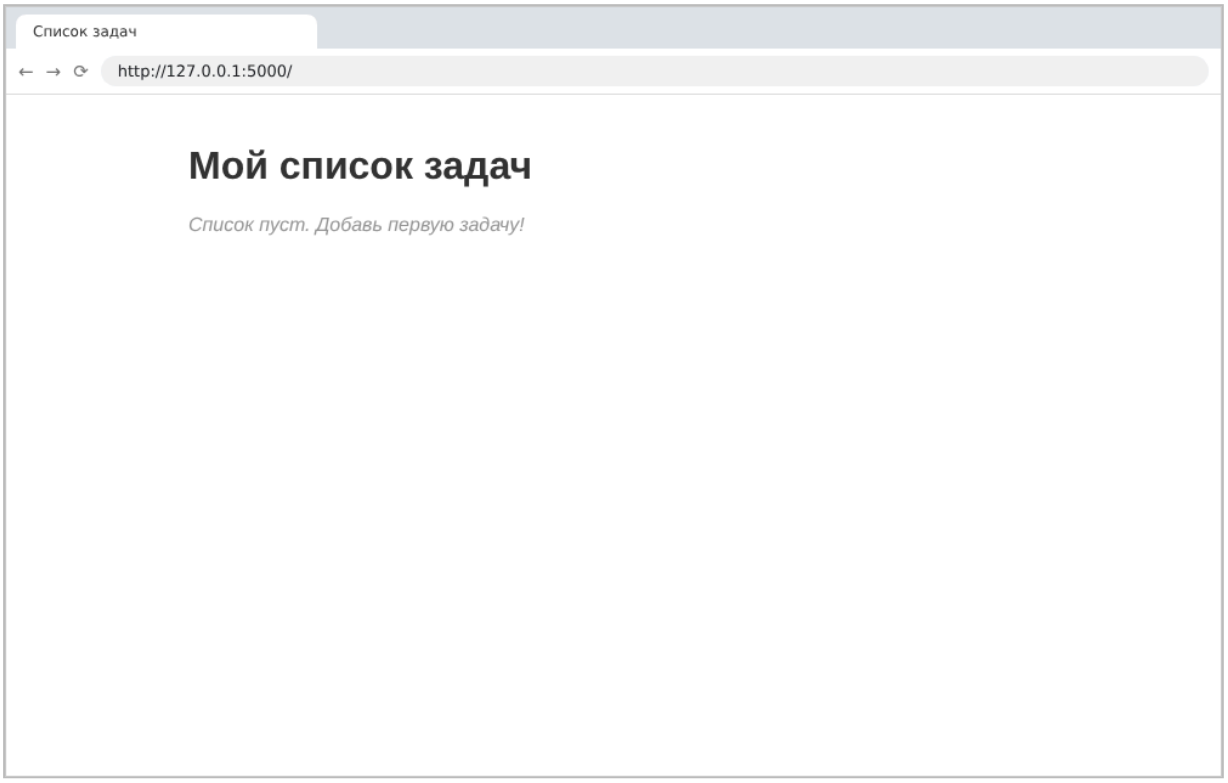



Рисунок 5 — Отображение пустого списка

5. Эксперимент: закомментировал строку conn.row_factory = sqlite3.Row. Страница загрузилась, но вместо названий и статусов выводятся пустые значения: строки стали обычными кортежами, у которых нет полей id, title и status. Jinja2 по умолчанию не выбрасывает исключение, а подставляет пустую строку (Undefined), поэтому ошибка «тихая». После возврата row_factory страница снова отображается правильно.

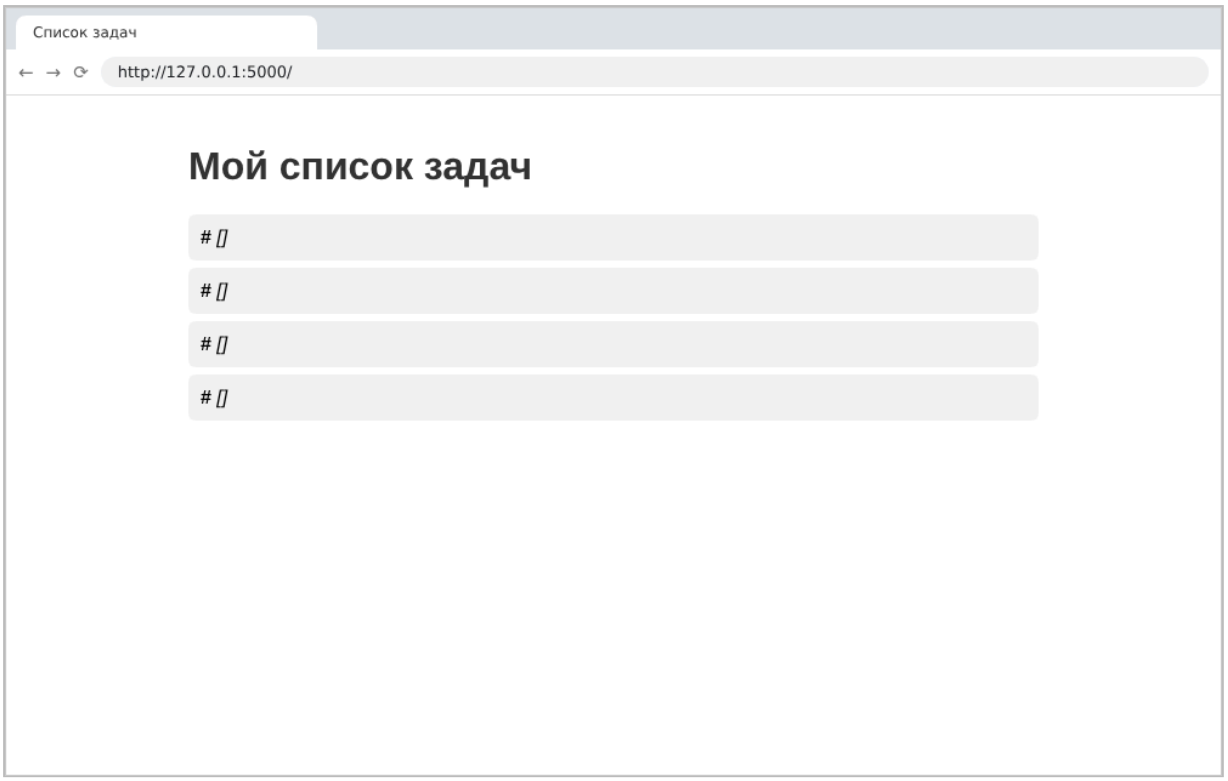



Рисунок 6 — Результат эксперимента без sqlite3.Row

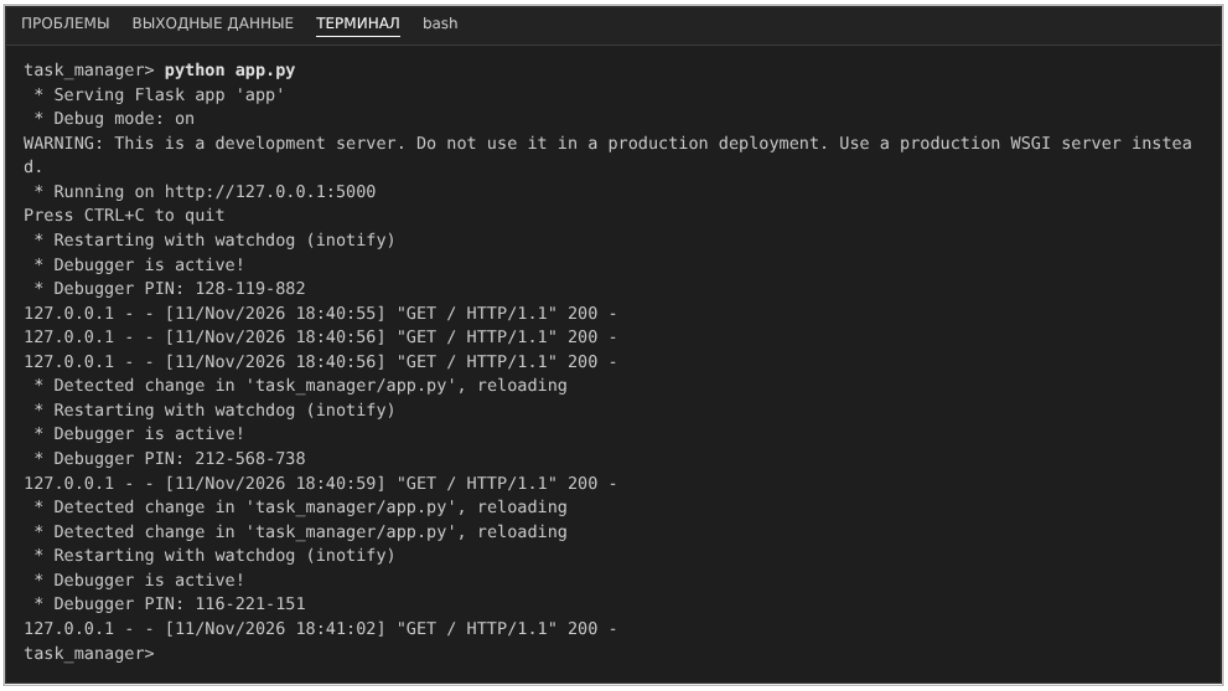



Рисунок 7 — Журнал сервера во время работы

## Примеры запросов и ответов

Таблица 1 — Примеры HTTP-запросов и ответов сервера

| **Запрос** | **Код ответа** | **Заголовок Location** | **Результат** |
|---|---|---|---|
| GET / | 200 OK | — | HTML-страница со списком задач из app.db |
| POST / | 405 METHOD NOT ALLOWED | — | Маршрут принимает только GET |

## Ответы на контрольные вопросы

### 1. Зачем нужен шаблонизатор Jinja2, если можно вернуть HTML-строку из функции?

Шаблонизатор отделяет оформление страницы от Python-логики: HTML хранится в отдельном файле, его удобно редактировать, редактор подсвечивает разметку, части страниц можно переиспользовать. Кроме того, Jinja2 автоматически экранирует выводимые значения, что защищает от внедрения HTML-кода.

### 2. Чем {{ ... }} отличается от {% ... %} в Jinja2?

{{ ... }} выводит значение выражения в HTML, а {% ... %} — управляющая конструкция (цикл for, условие if, блоки), которая сама ничего не выводит, а управляет тем, какая разметка попадёт в результат.

**3. Что делает render_template('index.html', items=rows)?**

Находит файл templates/index.html, передаёт в него переменную items со значением rows, обрабатывает шаблон Jinja2 и возвращает готовую HTML-строку, которую Flask отправляет браузеру.

**4. Почему при чтении данных не нужен conn.commit()?**

commit фиксирует изменения, а SELECT ничего не изменяет. Поэтому после чтения достаточно закрыть соединение.

**5. Что такое sqlite3.Row и почему item.title работает только с ним?**

sqlite3.Row — фабрика строк, при которой каждая строка результата поддерживает доступ и по индексу, и по имени колонки. Jinja2 при записи item.title ищет атрибут или ключ title. У обычного кортежа такого ключа нет, поэтому без row_factory значение не находится (что показал эксперимент).

**6. Куда Flask кладёт файлы шаблонов по умолчанию и можно ли это изменить?**

По умолчанию шаблоны ищутся в папке templates рядом с файлом приложения. Изменить путь можно параметром Flask(__name__, template_folder='другая_папка').

**7. Что выведет шаблон, если items — пустой список?**

Пустой список в Jinja2 считается ложным значением, поэтому выполнится ветка {% else %} и на странице появится сообщение «Список пуст. Добавь первую задачу!».

**Вывод**

В ходе работы я связал Flask-приложение с базой SQLite и вывел данные на страницу через шаблон Jinja2. Я убедился, что сервер и консольное приложение работают с одним файлом базы, проверил отображение пустого списка и на эксперименте понял, зачем нужен sqlite3.Row.